ESOFT METRO CAMPUS • FACULTY OF COMPUTING

BIT (Hons) in Software Engineering Degree Programme

DUTCH POINT NEGOMBO • HOTEL MANAGEMENT SYSTEM

Comprehensive System Demonstration & Viva Presentation Guide

> **PRO TIP / KEY TAKEAWAY: ACADEMIC INTERIM MILESTONE (55% - 60% COMPLETION)**
> This guide provides the exact spoken script, screen choreography, live database verification commands, and technical walkthrough for demonstrating the backend (ASP.NET Core 8), frontend (React SPA), and database (MySQL) during the Interim Viva examination. All currency calculations strictly reflect Sri Lankan Rupees (LKR).

Project Metadata & Presenting Team

Student Name	Student ID	Project Role in Viva	Key Demonstration Module
I.D.P.N Iddamalgoda	E269559	Lead Developer & System Admin	Architecture, Admin Dashboard & Concurrency Verification
H.M.Y.R Herath	E267614	Backend Software Engineer	ASP.NET Core Web API, Controllers & LINQ Overlap Algorithm
Y.M.A.Y Bandara	E267609	Frontend Software Engineer	React SPA, UI/UX, Dynamic Filters & Live Booking Modal
A.G.H.H Jinadasa	E267612	QA & Requirements Engineer	Requirements Traceability Matrix (RTM) & Test Case Execution

1. Pre-Demonstration Setup & Screen Preparation

Before inviting examiners to evaluate the demonstration, ensure that all services are active and four dedicated desktop windows are arranged for seamless switching (Alt+Tab):

Window / Tool	Target URL / Command	Display State	Purpose in Demonstration
Frontend UI	http://localhost:5173	Full Screen Browser	Primary demonstration of customer registration, search, and booking.
Swagger REST API	http://localhost:5000/swagger	Browser Tab 2	Shows RESTful endpoints, request/response models, and status codes.
MySQL Database	hotel_db in Workbench / phpMyAdmin	Split / Background	Proves live transactional persistence and zero orphaned records.
Visual Studio / VS	BookingsController.cs (Line 42-	Minimized / Ready	Code review of transactional

Code	65)	double-booking prevention algorithm.

1.1 Pre-Flight Health Check Commands (PowerShell)

Run these quick checks prior to the presentation to ensure ports and services are operating normally:

```
# 1. Verify MySQL Service (Port 3306)
Get-Service -Name MySQL*

# 2. Launch Backend API (Port 5000)
cd C:\Users\PASINDU\Desktop\Interim\backend
dotnet run --urls "http://localhost:5000"

# 3. Launch Frontend Client (Port 5173)
cd C:\Users\PASINDU\Desktop\Interim\frontend
npm run dev
```

2. Presentation Strategy & Time Allocation

The presentation is timed for an optimal 12 to 14 minute interim demonstration window:

Stage	Duration	Presenter	Primary Objective
1. Architecture & Tech Stack	1.5 Mins	Pasindu Nirmal	Introduce three-tier decoupled structure and interim progress.
2. Database Schema Inspection	2.0 Mins	Pasindu / Herath	Show relational integrity, LKR room pricing, and empty test slate.
3. Backend API & Concurrency	3.0 Mins	H.M.Y.R Herath	Explain C# controller logic and mathematical overlap formula.
4. Frontend Live Demonstration	5.0 Mins	Bandara & Pasindu	Live registration, date/capacity search, booking, and double-booking block.
5. Admin Operations & RTM	2.0 Mins	Pasindu & Jinadasa	Show Admin dashboard, revenue statistics, and RTM test alignment.
6. Examiner Q&A Defense	Flexible	All Team Members	Defend architectural decisions, security controls, and final roadmap.

3. Step-by-Step Live Demonstration Script

Phase 1: Architecture & Technology Introduction (1.5 Mins)

Spoken By: Lead Developer & System Administrator (Pasindu Nirmal)

"Good morning respected panel of lecturers. Today our team is presenting the Interim progress of the Dutch Point Negombo Web-Based Hotel Management System. As established in our interim deliverables, the system has reached 55% to 60% completion, covering the core reservation lifecycle, user authentication, and inventory administration.

Our system is architected as an enterprise Three-Tier solution:
• Presentation Tier: A responsive Single Page Application built with React and Vite.
• Application Tier: A secure, high-throughput REST API built on ASP.NET Core 8 and C#.
• Data Tier: A relational MySQL database enforcing referential integrity and optimized indexes via Entity Framework Core.

To provide a rigorous technical demonstration, we will not merely demonstrate clicking buttons on the UI; we will show you how each transaction originates in the browser, processes through the C# business logic, and mutates state in MySQL."

Phase 2: Database Layer Showcase (2 Mins)

Spoken By: Lead Developer (Pasindu) and Backend Engineer (Herath)

Choreography: Switch screen to MySQL Workbench / phpMyAdmin with database 'hotel_db' visible.

Action 1: 1. Run query: SELECT RoomId, RoomNumber, RoomType, PricePerNight, Capacity, Status FROM Rooms;

"Notice here in our database that all room prices are stored natively in Sri Lankan Rupees (LKR), ranging from LKR 28,000 for standard Deluxe Suites to LKR 140,000 for Presidential Ocean Villas. Each room has an enforced guest Capacity limit. This resolves the currency and capacity feedback raised during our early requirements gathering stage."

Action 2: 2. Run query: SELECT BookingId, RoomId, UserId, CheckInDate, CheckOutDate, Status FROM Bookings;

"Notice that Suite 101 currently has no reservations for the target test dates of October 10th to October 12th, 2026. We will now initiate a live transaction on the frontend and track this record into the database."

Phase 3: Backend REST API & Concurrency Logic (3 Mins)

Spoken By: Backend Software Engineer (H.M.Y.R Herath)

Choreography: Switch screen to Swagger UI (http://localhost:5000/swagger) and Visual Studio showing BookingsController.cs.

"In our ASP.NET Core application, we have separated operations into dedicated domain controllers: AuthController handles token generation and user registration; RoomsController manages inventory querying; and BookingsController encapsulates transactional reservation processing.

The primary engineering hurdle in hospitality software is the prevention of double bookings when multiple users attempt to reserve the same physical suite for overlapping dates. Here is our transactional check inside BookingsController:"

```
// Transactional Date Overlap Validation in C# / LINQ
var isOverlapping = await _context.Bookings.AnyAsync(b =>
    b.RoomId == request.RoomId &&
    b.Status != "Cancelled" &&
    b.CheckInDate < request.CheckOutDate &&
    b.CheckOutDate > request.CheckInDate
);

if (isOverlapping)
{
    return Conflict(new { message = "Room is already reserved for the selected dates." });
}
```

"This LINQ expression implements the standard mathematical interval intersection formula: a conflict exists if and only if an existing check-in is earlier than the requested check-out AND an existing check-out is later than the requested check-in. If true, the API rejects the request with HTTP 409 Conflict before any database write occurs."

Phase 4: Frontend Live Demonstration & End-to-End Workflow (5 Mins)

Spoken By: Frontend Software Engineer (Y.M.A.Y Bandara) & Pasindu Nirmal

Choreography: Switch browser to http://localhost:5173 in full-screen view.

Sub-Scenario A: Scenario A: Secure Registration & Role-Based Access Control (RBAC)

Action: Click 'Sign In' -> Click 'Register an Account'. Show the modal fields.

"Notice that our registration dialog contains fields for Full Name, Email, Password, Phone Number, and Address, but completely excludes any role selection dropdown. In our initial design, visitors could select their role. During security review, this was recognized as an OWASP Broken Access Control vulnerability. Now, our backend strictly hardcodes the 'Customer' role for all public registrations, completely preventing privilege escalation."

Live Action: Register user: Kasun Silva | Email: kasun@demo.com | Password: Kasun@123 | Phone: 0771234567 | Address: Negombo.

Result: Click 'Complete Registration'. System automatically logs Kasun in. The header shows 'Kasun Silva [Customer]'.

DB Proof: Switch to MySQL: Run SELECT UserId, FullName, Email, Role FROM Users WHERE Email='kasun@demo.com';. Show the lecturer that Kasun is stored with Role='Customer'.

Sub-Scenario B: Scenario B: Multi-Variable Search, Guest Count & LKR Pricing

Action: Navigate to the search bar. Select Check-In: 2026-10-10, Check-Out: 2026-10-12 (2 Nights). Select Guest Count: 2 Guests.

Result: Click 'Check Rates'. Point out the green 'AVAILABLE' badge on Deluxe Suite 101 and the rate: LKR 28,000 / Night.

"The system automatically calculates the stay total: 2 nights at LKR 28,000 per night equals LKR 56,000.
Now, watch what happens if we change the Guest Count to 5 Guests. Deluxe Suite 101 (maximum capacity 2) disappears from the search results, and only larger Family Villas with Capacity >= 5 are displayed. This eliminates guest overcapacity errors."

Sub-Scenario C: Scenario C: Live Booking Creation & Instant Database Persistence

Action: Reset Guest Count to 2 Guests. Click 'Book Now' on Deluxe Suite 101.

Action: Review the booking summary modal: 2 Guests, 2 Nights, Total Stay: LKR 56,000. Click 'Confirm Booking'.

Result: A confirmation dialog appears: 'Reservation Confirmed! Booking ID: RES-1001'.

DB Proof: Switch to MySQL: Run SELECT BookingId, RoomId, UserId, TotalAmount, Status FROM Bookings ORDER BY BookingId DESC LIMIT 1;. Show the newly created row with TotalAmount=56000.00 and Status='Confirmed'.

Sub-Scenario D: Scenario D: Concurrency Test - Live Overlapping Double-Booking Prevention

Action: Stay in the browser. Select dates that overlap with Kasun's booking: 2026-10-11 to 2026-10-14 for Deluxe Suite 101.

Action: Click 'Book Now' -> 'Confirm Booking'.

Result: The UI displays an immediate red warning alert: 'Room is already reserved for the selected dates. Please choose another date or room.'

Inspector Proof: Press F12 -> Network Tab -> Show the POST /api/Bookings request returning red HTTP 409 Conflict from ASP.NET Core.

"This live test proves that our backend actively enforces transactional concurrency, protecting the resort from costly double-booking errors."

Phase 5: Admin Dashboard & Operational Controls (2 Mins)

Spoken By: Lead Developer & System Administrator (Pasindu Nirmal)

Action: Log out from Kasun's account. Click 'Sign In' and enter Admin credentials: Username: Pasindu | Password: Pasindu@123.

Result: Point out the top navigation bar. Notice that 'Admin Dashboard' and 'Inventory Management' are now accessible.

"Upon logging in with the administrative identity, the navigation dynamically adapts. Inside the Admin Dashboard:
• Operational KPI Cards: Displays Total Revenue (in LKR), Total Active Bookings, and Available vs Occupied Room counts.
• Occupied Rooms Tracker: Suite 101 is clearly flagged as Occupied across October 10th to 12th.
• Reservation Roster: Front-desk staff can inspect guest contact details, verify headcount, and execute manual check-ins or cancellations."

Phase 6: Traceability Matrix (RTM) & Test Verification (1.5 Mins)

Spoken By: QA & Requirements Engineer (A.G.H.H Jinadasa)

"Every feature demonstrated today links directly back to the 5-stage Requirements Engineering process defined in Chapter 2 of our Interim Report:
• Business Objective 1 (Automated Reservations) was verified via Test Case TC-01.
• Business Objective 2 (Double-Booking Prevention) was verified via Test Case TC-04 (HTTP 409 Conflict).
• Business Objective 3 (Role-Based Access Control) was verified via Test Case TC-05.
• Business Objective 4 (Capacity & Headcount Enforcement) was verified via Test Case TC-09.

All test cases executed live during this session have passed verification without defect. We are now on schedule to complete the remaining 40% of the project for the final viva."

4. Lecturer Viva Q&A Defense (Examiner Cheat Sheet)

Prepare for these common technical questions asked by software engineering panels:

Q1: Why did you choose ASP.NET Core 8 Web API instead of Node.js / Express or PHP?
Answer: ASP.NET Core provides enterprise-grade asynchronous throughput, compile-time type safety via C#, and native dependency injection. Furthermore, Entity Framework Core offers a robust Object-Relational Mapper (ORM) that natively parameterizes queries, completely eliminating SQL injection and managing transactional concurrency much more reliably than standard scripting languages.

Q2: How does your system protect against SQL Injection and Data Tampering?
Answer: We do not concatenate raw SQL strings anywhere in our backend. All database queries use LINQ expressions compiled into parameterized SQL by EF Core. On the authentication layer, passwords are protected, and public registration endpoints strictly ignore any client-supplied role payloads, hardcoding the role to 'Customer' on the server.

Q3: Why was currency converted from US Dollars ($) to Sri Lankan Rupees (LKR)?
Answer: Dutch Point Negombo is a boutique beachfront resort operating in Western Province, Sri Lanka. During our requirements gathering interview with the General Manager, management emphasized that front-desk operations and bank reconciliation occur strictly in Sri Lankan Rupees. Showing USD rates created a disconnect for domestic guests and required volatile exchange rate conversions. All pricing was permanently localized to LKR (LKR 28,000 to LKR 140,000 / night).

Q4: How does the system handle high traffic if two users click 'Book Now' at the exact same millisecond?
Answer: Our double-booking prevention algorithm executes inside an ACID database transaction. If two simultaneous HTTP POST requests arrive, the database transaction lock ensures the first request is committed while the second request evaluates the overlap condition, encounters the freshly committed row, and is gracefully rejected with HTTP 409 Conflict.

Q5: What exact features remain to be implemented for the Final (100%) submission?
Answer: The remaining 40% of the project scope consists of four well-defined modules: (1) Online Payment Gateway (IPG) integration via PayHere sandbox; (2) Automated PDF invoice and receipt generation; (3) Automated SMS/Email reservation confirmation alerts; and (4) Advanced historical occupancy and revenue predictive analytics for resort management.

5. Emergency Troubleshooting Checklist

If any technical issue arises during the presentation, stay calm and follow these quick recovery steps:

Potential Issue	Root Cause	Quick Fix (Run in PowerShell / Terminal)
Port 5000 Already in Use	Previous backend process still running	Get-Process -Name backend | Stop-Process -Force
MySQL Connection Refused	MySQL Windows Service stopped	Start-Service MySQL80 (or open XAMPP and click Start on MySQL)
Frontend Fails to Connect	CORS policy or backend offline	Verify backend terminal displays 'Now listening on: http://localhost:5000'
Overlapping Booking Not	Status was set to 'Cancelled'	Check MySQL Bookings table; cancelled reservations

Blocked	deliberately free up dates